pfi

# Python for Fixed Income

COURSE 2 · BOND MATH

# Portfolio Aggregation and Scenarios

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The desk lead's note: the book's top lines, and what-ifs

### The Desk Lead's Note

*The September book, 200 positions. I want the risk report's top lines from your own tested functions: value, duration, spread, DTS, convexity, DV01. Then the what-ifs, repriced in full: every yield -100, -25, +25 and +100 bp; every spread +25 bp and +10 percent. How far off is the duration-and-convexity shortcut? Which sectors carry each loss? Send me the grid as a workbook.*

The issuers are invented; every scenario is a what-if, never a forecast.

### Read the Book

```
holdings = pd.read_csv('holdings.csv')
holdings['as_of_date'] = pd.to_datetime(holdings['as_of_date']).dt.date
holdings['maturity'] = pd.to_datetime(holdings['maturity']).dt.date
total_value = holdings['market_value'].sum()
print(f'{len(holdings)} positions, {total_value:,.2f} dollars')
holdings[['cusip', 'issuer', 'sector', 'price', 'accrued', 'yield_pct', 'oas', 'par_held',
          'market_value']].head()
200 positions, 490,819,215.00 dollars
```

```
...   [100%]
37 passed in 0.50s
```

The module holds 23 functions and 37 tests; today adds two functions and three tests.

**First a function that weights the book the way Excel does.**

# 14.1 weighted_average, docstring first

"market_value" is par x (price + accrued) / 100, the dirty price; notebook 10 tied all 200 to the cent. A book's duration weights each bond's by its market value, as the Course 1 project did: sum of value x weight, over the sum of the weights. Course 1's Combining and Loading, Part 2, drew why the divide is by what the weights add to.

```
%%writefile -a bondmath.py
def weighted_average(values: list[float], weights: list[float]) -> float:
    if len(values) != len(weights):
        raise ValueError(f"{len(values)} values but {len(weights)} weights")
    total_weight = sum(weights)
    if total_weight == 0:
        raise ValueError("the weights sum to zero")
    total = 0.0
    for value, weight in zip(values, weights):
        total += value * weight
    return total / total_weight
```

"zip" walks the two lists in step, as in Course 1. The module takes plain lists, not pandas, so "list(...)" turns a column into one.

```
importlib.reload(bondmath)
book_duration_file = bondmath.weighted_average(list(holdings['duration']),
                                               list(holdings['market_value']))
excel_duration = 6.254777411342787
print(f'weighted_average {book_duration_file:.15f}, Excel {excel_duration:.15f}')
print(f"plain average    {holdings['duration'].mean():.6f} years")
weighted_average 6.254777411342787, Excel 6.254777411342787
plain average   6.470405 years
```

The plain average counts every position once, whatever its size. The larger positions here are shorter, so the weighted number is lower.

# Against Excel, an empty group, and two tests

Excel side by side. "holdings.csv" opened in Excel: I "price", K "accrued", N "duration", R "par_held", S "market_value".

| Job | Excel | Result |
|---|---|---|
| Bezane's market value (row 2) | `=R2*(I2+K2)/100` | 5,435,500.00 |
| Book value | `=SUMPRODUCT(R2:R201,I2:I201+K2:K201)/100` | 490,819,215.00 |
| Book duration | `=SUMPRODUCT(N2:N201,S2:S201)/SUM(S2:S201)` | 6.254777411342787 |
| Plain average | `=AVERAGE(N2:N201)` | 6.470405 |

```
try:
    bondmath.weighted_average([4.0, 6.0], [0, 0])
except ValueError as error:
    print(f'ValueError: {error}')
ValueError: the weights sum to zero
```

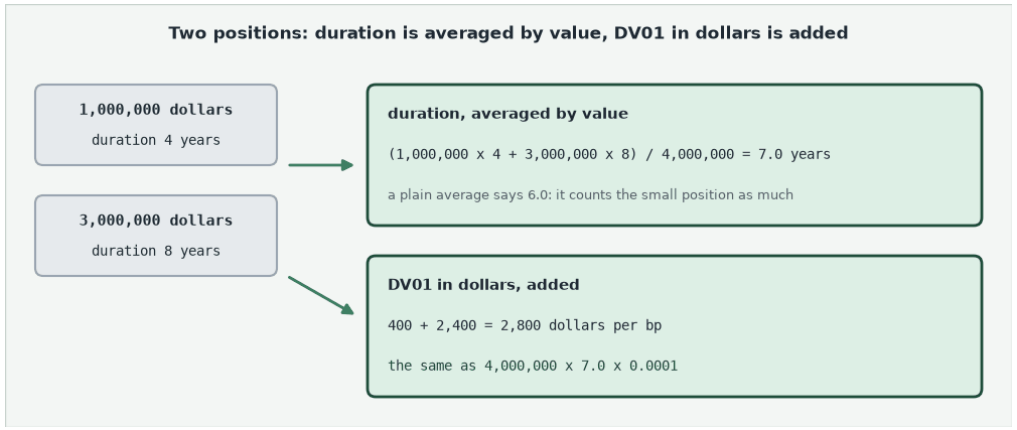
A group with no value in it has no average; the function says so instead of dividing by zero.

```
%%writefile -a test_bondmath.py
def test_weighted_average_known_case():
    assert bondmath.weighted_average([4.0, 6.0, 10.0], [1, 1, 2]) == pytest.approx(7.5, abs=1e-12)
def test_weighted_average_raises_on_zero_weights():
    with pytest.raises(ValueError):
        bondmath.weighted_average([4.0, 6.0], [0, 0])
```

```
...   [100%]
39 passed in 0.50s
```

**"weighted_average" returns Excel's SUMPRODUCT over SUM to the last digit: the book's duration is 6.254777 years.**

# 14.2 The book's risk card: averages and a sum




```
bonds = list(zip(holdings['as_of_date'], holdings['maturity'], holdings['coupon'],
                 holdings['yield_pct']))
holdings['dirty_bm'] = [bondmath.dirty_price(*bond) for bond in bonds]
holdings['modified_bm'] = [bondmath.modified_duration(*bond) for bond in bonds]
holdings['convexity_bm'] = [bondmath.convexity(*bond) for bond in bonds]
holdings['dts_bm'] = [bondmath.dts(bondmath.spread_duration(*bond), oas)
                      for bond, oas in zip(bonds, holdings['oas'])]
```

```
holdings['dv01_bm'] = [bondmath.dv01(*bond) for bond in bonds]
holdings['dv01_dollars'] = holdings['dv01_bm'] * holdings['par_held'] / 100
```

Some of a book's numbers are averages weighted by value; DV01 in dollars is a sum. Both come from the same bonds, so they agree.

| cusip | dirty_bm | modified_bm | convexity_bm | dts_bm | dv01_dollars |
|---|---|---|---|---|---|
| 99080CAE8 | 108.70 | 4.62 | 26.35 | 680.16 | 2514.98 |
| 99049XAE2 | 101.23 | 2.16 | 6.13 | 1158.15 | 54.78 |
| 99039NAC0 | 100.39 | 3.24 | 12.84 | 497.23 | 489.41 |
| 99039NAG1 | 95.17 | 12.11 | 232.88 | 2640.58 | 576.38 |
| 99034IAD4 | 95.13 | 7.33 | 66.38 | 931.91 | 1745.16 |

# The book in six numbers

```
weights = list(holdings['market_value'])
book_duration = bondmath.weighted_average(list(holdings['modified_bm']), weights)
book_spread = bondmath.weighted_average(list(holdings['oas']), weights)
book_dts = bondmath.weighted_average(list(holdings['dts_bm']), weights)
book_convexity = bondmath.weighted_average(list(holdings['convexity_bm']), weights)
book_dv01 = holdings['dv01_dollars'].sum()
```

```
market value  490,819,215.00 dollars
duration   6.2548 years
spread   180.46 bp
DTS   1064.11 years x bp
DV01   306,996.11 dollars per bp
```

DV01 is notebook 10's number; value x duration x 0.0001 gives it to within 0.10 dollars, as that notebook's Predict found.

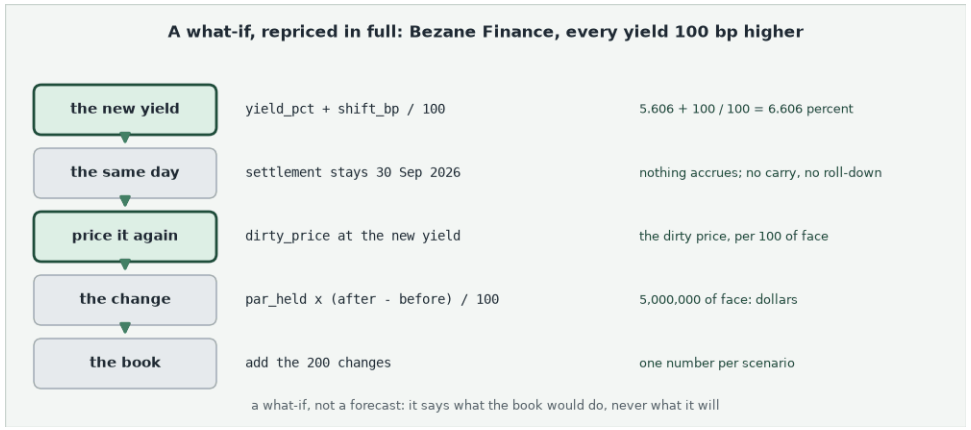
Excel side by side. Columns as in 14.1, with M "oas" and P "dts".

| Job | Excel | Result |
|---|---|---|
| Book spread | `=SUMPRODUCT(M2:M201,S2:S201)/SUM(S2:S201)` | 180.4609519769922 |
| Book DTS, the file's column | `=SUMPRODUCT(P2:P201,S2:S201)/SUM(S2:S201)` | 1064.113487036668 |
| Book DV01, the file's durations | `=SUMPRODUCT(N2:N201,S2:S201)/10000` | 306,996.49 |

Your own DTS, 1064.11, differs from the file column's by its rounding (notebook 13). Excel's DV01 reads the file's durations, rounded to 3 decimals: 0.38 dollars above your own.

**Four of the six are averages weighted by value and DV01 is a sum: this card is the base every what-if moves from.**

# 14.3 Every yield moves, repriced in full




A scenario is a mechanical what-if, not a forecast: every yield moved at once, today, and every bond priced again. A parallel shift moves every yield by the same bp. A full reprice runs "dirty_price" at each new yield; no estimate stands in for it.

```
base_value = (holdings['par_held'] * holdings['dirty_bm'] / 100).sum()
print(f'book from the yields  {base_value:,.2f} dollars')
print(f"book from the file    {total_value:,.2f} dollars, {total_value - base_value:,.2f} more")
book from the yields  490,819,038.14 dollars
book from the file   490,819,215.00 dollars, 176.86 more
```

The file's price and accrued are rounded to 3 decimals; every change below is measured from "base_value", so the rounding never enters one.

```
settlement, maturity, coupon_pct, yield_pct = bonds[0]
before = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct)
after = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct + 100 / 100)
change_dollars = holdings.loc[0, 'par_held'] * (after - before) / 100
print(f'dirty {before:.4f} at {yield_pct} percent, {after:.4f} at {yield_pct + 1:.3f}')
print(f'the position changes by {change_dollars:,.2f} dollars')
dirty 108.7099 at 5.606 percent, 103.8202 at 6.606
the position changes by -244,484.16 dollars
```

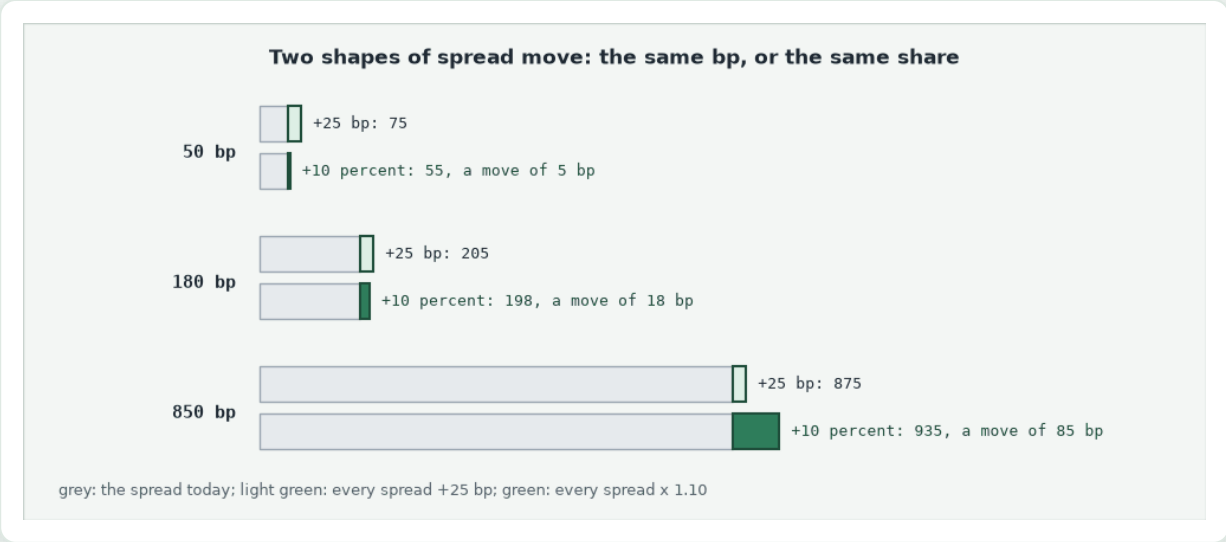
```
def reprice(new_yields_pct):
    after = [bondmath.dirty_price(s, m, c, y) for s, m, c, y in
             zip(holdings['as_of_date'], holdings['maturity'], holdings['coupon'], new_yields_pct)]
    return holdings['par_held'] * (pd.Series(after) - holdings['dirty_bm']) / 100
```

```
by_dv01 = 25 * book_dv01
average_25 = (totals['rates -25 bp'] - totals['rates +25 bp']) / 2
print(f'25 x book DV01            {by_dv01:,.2f} dollars')
print(f'the two reprices averaged {average_25:,.2f} dollars')
25 x book DV01   7,674,902.76 dollars
the two reprices averaged 7,676,201.59 dollars
```

DV01 is the slope at today's yields; for 25 bp it is within 1,300 dollars of the reprice.

| Job | Excel |
|---|---|
| A bond at the shifted yield (V2) | =PRICE(A2,H2,G2/100,(L2+$Y$1/100)/100,100,2,0) |
| Its value (W2) | =R2*(V2+K2)/100 |

# 14.4 Every spread moves: two shapes




Notebook 13's DTS source found that wider spreads move more, so a desk runs a second shape. A proportional spread move takes each spread times 1.10: the wide bonds move most. Both shapes are mechanical what-ifs, not forecasts.

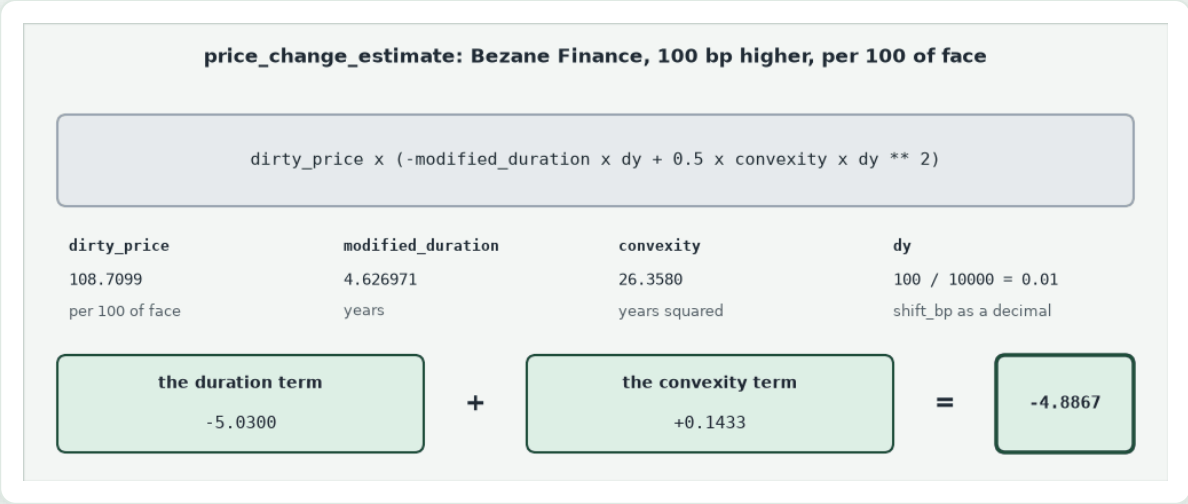
```
pnl['spreads +10 percent'] = reprice(holdings['yield_pct'] + 0.10 * holdings['oas'] / 100)
spread_10 = pnl['spreads +10 percent'].sum()
print(f'spreads +10 percent: {spread_10:,.2f} dollars, {spread_10 / base_value * 100:.4f} percent')
spreads +10 percent: -5,150,093.57 dollars, -1.0493 percent
```

The book's spread is 180.46 bp, so its average move is about 18 bp: less than 25, and so a smaller loss.

Notebook 13's rule: a spread 10 percent wider moves the price about -DTS x 0.10 / 100 percent.

```
from_dts = -book_dts * 0.10 / 100
print(f'from book DTS  {from_dts:.4f} percent')
print(f'full reprice   {spread_10 / base_value * 100:.4f} percent')
from book DTS  -1.0641 percent
full reprice   -1.0493 percent
```

# 14.5 price_change_estimate, against the full reprice




Notebook 10 built this estimate from two terms; now it goes in the module. The pieces: the dirty price per 100, modified duration in years, convexity in years squared, and dy = shift_bp / 10000.

```
%%writefile -a bondmath.py
def price_change_estimate(dirty_price: float, modified_duration: float, convexity: float, shift_bp: float) -> float:
    dy = shift_bp / 10000
    return dirty_price * (-modified_duration * dy + 0.5 * convexity * dy ** 2)
```

```
importlib.reload(bondmath)
bezane = holdings.iloc[0]
modified, convexity = bezane['modified_bm'], bezane['convexity_bm']
with_convexity = bondmath.price_change_estimate(before, modified, convexity, 100)
duration_only = bondmath.price_change_estimate(before, modified, 0.0, 100)
print(f'full reprice            {after - before:.4f} per 100')
print(f'duration and convexity  {with_convexity:.4f} per 100')
print(f'duration alone          {duration_only:.4f} per 100')
full reprice   -4.8897 per 100
duration and convexity  -4.8867 per 100
duration alone   -5.0300 per 100
```

Duration alone misses by 0.14 per 100; the convexity term takes the miss to 0.003.

At 5 bp the estimate should match a full reprice of every holdings row of the reference file to 0.0001 per 100. The test keeps those rows with a comprehension's "if", as Course 1 filtered lists.

```
%%writefile -a test_bondmath.py
def test_estimate_close_to_full_reprice_for_small_shift():
    shift_bp = 5
    rows = [row for row in REFERENCE if row["group"] == "holdings"]
    for row in rows:
        settlement, maturity, coupon_pct, yield_pct, frequency, basis = bond_inputs(row)
        before = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
        after = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct + shift_bp / 100, frequency, basis)
        duration_years = bondmath.modified_duration(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
        convexity_years2 = bondmath.convexity(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
        estimate = bondmath.price_change_estimate(before, duration_years, convexity_years2, shift_bp)
        assert estimate == pytest.approx(after - before, abs=1e-4), row["case_id"]
```

```
...   [100%]
40 passed in 0.50s
```

# The estimate for the book, at 25 bp

"estimate" is reprice's twin: the shortcut for every bond, at each bond's own shift in bp.

```
def estimate(shifts_bp):
    per_100 = [bondmath.price_change_estimate(p, d, c, s) for p, d, c, s in
               zip(holdings['dirty_bm'], holdings['modified_bm'], holdings['convexity_bm'],
                   shifts_bp)]
    return holdings['par_held'] * pd.Series(per_100) / 100
```

### The 25 bp Moves

```
for shift_bp in [-25, 25]:
    full = totals[f'rates {shift_bp:+d} bp']
    miss = full - estimate([shift_bp] * len(holdings)).sum()
    print(f'{shift_bp:+d} bp: full {full:,.2f}, the shortcut misses by {miss:,.2f} dollars')
-25 bp: full 7,778,969.05, the shortcut misses by 1,314.62 dollars
+25 bp: full -7,573,434.14, the shortcut misses by -1,283.05 dollars
```

# 14.6 Where each loss lands, by sector

A weighted average splits into sums, so "groupby" can add each piece by sector and divide after. "duration" is the file's, as the Excel pivot reads it.

```
holdings['mv_x_duration'] = holdings['market_value'] * holdings['duration']
holdings['mv_x_dts'] = holdings['market_value'] * holdings['dts_bm']
sums = holdings.groupby('sector')[['market_value', 'mv_x_duration', 'mv_x_dts',
                                   'dv01_dollars']].sum()
assert abs(sums['market_value'].sum() - total_value) < 0.005
assert abs(sums['dv01_dollars'].sum() - book_dv01) < 1e-6
```

```
by_sector = pd.DataFrame({'weight_pct': sums['market_value'] / total_value * 100,
                          'duration': sums['mv_x_duration'] / sums['market_value'],
                          'dts': sums['mv_x_dts'] / sums['market_value'],
                          'dv01_share_pct': sums['dv01_dollars'] / book_dv01 * 100})
print(f"Transportation's duration {by_sector.loc['Transportation', 'duration']:.12f} years")
by_sector.sort_values('weight_pct', ascending=False).round(2)
Transportation's duration 8.186803162389 years
```

| sector | weight_pct | duration | dts | dv01_share_pct |
|---|---|---|---|---|
| Materials | 14.61 | 5.26 | 863.68 | 12.29 |
| Financials | 13.90 | 6.22 | 1006.25 | 13.82 |
| Consumer | 11.54 | 6.97 | 1395.78 | 12.86 |
| Telecom | 11.38 | 6.16 | 1090.25 | 11.20 |
| Industrials | 10.70 | 6.70 | 1567.20 | 11.46 |
| Transportation | 9.95 | 8.19 | 914.17 | 13.02 |
| Health Care | 7.93 | 5.13 | 637.25 | 6.50 |
| Energy | 7.91 | 6.06 | 1025.17 | 7.67 |
| Technology | 6.18 | 5.47 | 869.87 | 5.40 |
| Utilities | 5.91 | 6.12 | 1166.57 | 5.78 |

A helper column "mv_x_duration", "=S2*N2", filled down, then a PivotTable: "sector" in Rows, Sum of "market_value", and a calculated field "=mv_x_duration/market_value".

| Job | Excel | Result |
|---|---|---|
| Transportation's duration | the calculated field's cell | 8.186803162389122 |
| The whole book | Grand Total | 6.254777411342787 |

The pivot equals "groupby" in all ten sectors, within 1e-15 years.

# 14.7 The grid to the desk, the code to Git

```
with pd.ExcelWriter('scenario_grid.xlsx') as writer:
    grid.round(2).to_excel(writer, sheet_name='scenarios')
    sector_pnl.round(2).to_excel(writer, sheet_name='by_sector')
```

```
back = pd.read_excel('scenario_grid.xlsx', sheet_name=None, index_col=0)
assert np.isclose(back['scenarios'], grid.round(2)).all()
assert np.isclose(back['by_sector'], sector_pnl.round(2)).all()
print(list(back.keys()), 'read back, every number as written')
['scenarios', 'by_sector'] read back, every number as written
```

```
!git -C "{work}" status --short
 M bondmath.py
 M test_bondmath.py
?? scenario_grid.xlsx
```

" M" beside the two files you appended to; "??" beside the workbook, which Git has never seen.

```
!git -C "{work}" add bondmath.py test_bondmath.py scenario_grid.xlsx
!git -C "{work}" status --short
M  bondmath.py
A  scenario_grid.xlsx
M  test_bondmath.py
```

"A" means staged as a new file: the workbook would go into the commit. It is output, made again by running the notebook; the record keeps code. "git restore --staged" takes a file out of the staging area and leaves it in the folder, unlike notebook 04's "git restore", which undoes the edit itself.

```
!git -C "{work}" restore --staged scenario_grid.xlsx
!git -C "{work}" status --short
M  bondmath.py
M  test_bondmath.py
?? scenario_grid.xlsx
```

```
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" show --stat --oneline HEAD
1b74708 Notebook 14: weighted_average and price_change_estimate
 bondmath.py    | 33 +++++++++++++++++++++++++++++++++
 test_bondmath.py | 26 ++++++++++++++++++++++++++
 2 files changed, 59 insertions(+)
```

**The desk gets the workbook; the record gets the 59 lines that make it.**

# Bring it home, and what you can now do

## You Can Now

Weight a book by market value with "weighted_average", as Excel's "SUMPRODUCT" over "SUM" does.

Build a book's risk card from your own functions, and say which numbers are averages and which add up.

Reprice a whole book in full under parallel shifts and under two shapes of spread move, and say what a scenario is and is not.

Estimate the same moves with "price_change_estimate", and say when the shortcut stops being enough.

Split each scenario's loss by sector, with "groupby" or a PivotTable, and tie the parts back to the book.

Send a scenario grid as a workbook, and take a file out of the staging area with "git restore --staged".

Copy today's two files into your own "my_bondmath" folder, then test and commit there. The copy replaces your files: anything of your own must be in the work folder first. In PowerShell, with the course's ".venv" active, and your eight characters from the setup cell in place of "k4w7q2xr":

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_14_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py", "$work\.gitignore" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py
git status --short
git commit -m "Notebook 14: weighted_average and price_change_estimate" -m "Notebook 14 rolls the bonds up into a book and reprices it under rate and spread what-ifs"
```

If a file slipped in, "git restore --staged " takes it out before the commit. In Colab, download the two files, and the workbook if you want it, from the work folder in the file browser.

# Predict first: 100 bp in full, and spreads against rates

## Predict 1

The book's DV01 is 306,996.11 dollars per bp. Every yield rises 100 bp at once. What does the full reprice take off the book, in millions of dollars? Write down a number.

```
pnl = pd.DataFrame()
for shift_bp in [-100, -25, 25, 100]:
    pnl[f'rates {shift_bp:+d} bp'] = reprice(holdings['yield_pct'] + shift_bp / 100)
totals = pnl.sum()
print(f"a 100 bp fall gains {totals.iloc[0] + totals.iloc[3]:,.2f} dollars more than a rise loses")
totals.round(2)
```

## Predict 2

Every spread up 25 bp, against 14.3's every yield up 25 bp, bond by bond:

A) the same to the cent: both move every yield 25 bp

B) a few dollars apart: the curve stays put

C) thousands apart: spread duration is not duration

```
pnl['spreads +25 bp'] = reprice(holdings['yield_pct'] + 25 / 100)
gap = (pnl['spreads +25 bp'] - pnl['rates +25 bp']).abs().max()
print(f"spreads +25 bp: {pnl['spreads +25 bp'].sum():,.2f} dollars")
print(f'largest difference from rates +25 bp, over 200 bonds: {gap:.2f} dollars')
```

# Predict first: the shortcut at 100 bp, and a sector's share

## Predict 3

At 25 bp the shortcut misses the full reprice by about 1,300 dollars. At 100 bp, four times the move, by how many dollars? Write down a number.

```
shifts = pd.DataFrame({'rates -100 bp': -100, 'rates -25 bp': -25, 'rates +25 bp': 25,
                       'rates +100 bp': 100, 'spreads +25 bp': 25,
                       'spreads +10 percent': 0.10 * holdings['oas']})
grid = pd.DataFrame({'full_dollars': pnl.sum()})
grid['full_pct'] = grid['full_dollars'] / base_value * 100
grid['estimate_dollars'] = [estimate(shifts[name]).sum() for name in grid.index]
grid['miss_dollars'] = grid['full_dollars'] - grid['estimate_dollars']
grid.round(2)
```

## Predict 4

Transportation is 9.9 percent of the value, with the longest duration, 8.19 years, and a DTS of 914 against the book's 1,064. Its share of the loss when every spread widens 10 percent is:

A) above 9.9 percent

B) about 9.9 percent

C) below 9.9 percent

```
sector_pnl = pnl.groupby(holdings['sector']).sum()
assert np.isclose(sector_pnl.sum(), pnl.sum()).all()
shares = sector_pnl / sector_pnl.sum() * 100
compare = pd.DataFrame({'value': by_sector['weight_pct'],
                        'rates +100 bp': shares['rates +100 bp'],
                        'spreads +10 percent': shares['spreads +10 percent']})
compare.loc[['Transportation', 'Consumer', 'Industrials']].round(2)
```

# The note, answered

| The note asked | Answer |
|---|---|
| The top lines | 490,819,215.00 dollars; duration 6.2548 years; spread 180.46 bp; DTS 1064.11; convexity 66.99; DV01 306,996.11 dollars per bp |
| Every yield moved | -100 bp +32.43 million; -25 bp +7.78; +25 bp -7.57; +100 bp -29.13 |
| Every spread moved | +25 bp -7.57 million; +10 percent -5.15 million |
| The shortcut | within 1,315 dollars at 25 bp; 87,346 at 100 bp |
| Which sectors | rate losses follow DV01, proportional spread losses follow DTS |
| The workbook | scenario_grid.xlsx, sheets scenarios and by_sector |

Be proud: a book's risk report, what-ifs included, from functions you wrote and tested.

**Next is the Course 2 Capstone: a new book arrives as a raw extract with a cover note, and every tool of the last three weeks gets used.**